

Assignment 1 - Shover Project

Artificial Intelligence

Hasti Javadzadeh-Farzaneh Khalili

During Phase 1 of our project, we built the core functionality of the Shover World environment. We implemented the entire environment logic inside a single class called `ShoverWorldEnv`, which manages everything that happens on the grid, including movement, pushing mechanics, box destruction, stamina updates, map handling, and perfect-square detection. The grid uses a numerical encoding system where empty cells, boxes, barriers, and lava are represented with specific integer values. The agent itself is tracked separately through its coordinates on the grid.

We designed the movement system so that the agent can move in four directions and push chains of boxes. When the agent pushes a line of boxes, the chain propagates forward correctly and interacts with the environment rules. Boxes pushed into lava get destroyed, boxes pushed into the edge of the grid are also destroyed, as a lava line spans the entire boundary, and boxes pushed into barriers are blocked. We also implemented stamina mechanics, where each timestep applies a baseline cost and any push action applies additional cost depending on the length of the push chain.

We implemented direction-specific stationary and non-stationary behavior for boxes: pushing a box makes it non-stationary in that direction for the next timestep, and if it does not move afterward, it becomes stationary again. This status is tracked per direction and is used directly in the push-cost calculation, an initial force is applied only when the box is stationary in that direction; otherwise, no initial force is needed.

Destroying boxes in lava refunds stamina based on the agent's initial force. Each time a box is destroyed in a lava cell, the total reward is increased accordingly. These mechanics follow the rules specified in our assignment.

We added two special abilities, barrier-maker and hellify. Both depend on detecting "perfect squares" of boxes on the grid. The barrier-maker converts the oldest perfect square into barriers and increases stamina based on the size of the square. Hellify is more destructive: it transforms a perfect square of size three or larger by turning its interior into lava and its border into empty cells. These special actions are managed by a separate class inside the environment called the `PerfectSquareManager`.

The PerfectSquareManager is responsible for identifying all valid perfect squares of boxes, tracking their ages over time, and dissolving them automatically if they remain unchanged for too long. The detection checks that a candidate region consists only of boxes and that no other boxes touch the perimeter around the square. When a perfect square remains in place across steps, its age increases. When it reaches a defined threshold, the square dissolves by removing its boxes from the grid. The manager stores this information and exposes it to the environment so that special actions know which square to operate on.

We also created a fully interactive graphical interface using pygame. In this interface, the grid is drawn visually with different colors representing empty spaces, boxes, barriers, and lava.

The agent is drawn as a blue circle. As the agent moves and interacts with the grid, the GUI updates in real time. Perfect squares detected on the grid are highlighted with a green border. We added keyboard controls so the user can move the agent with WASD keys and trigger special actions using B for barrier-maker and H for hellify. The user can also click on an empty grid cell to teleport the agent to that location. We included support for resetting the environment with the R key and quitting the game with Q. The GUI also displays useful information at the bottom of the screen, such as the current timestep, stamina, agent position, and also total reward.

For map handling, we implemented support for loading maps from files in both numeric and symbolic formats. The environment can read .txt files describing custom layouts, and if no map is provided, the environment generates a random grid using a specified number of boxes, barriers, and lava cells. Map validation ensures that formats are correct and that the agent is placed on a valid cell.

We added a termination condition so that the game ends automatically when there are no boxes left on the grid, or when stamina is depleted, or the maximum number of steps has been reached. The reset method restores the environment to its initial state, including the original map, stamina, timestep, perfect square ages, and agent position.

Finally, we prepared and integrated the unit testing structure in this phase. This framework supports tests for push-chain behavior, stamina calculations, perfect-square detection, invalid actions, map loading, and special abilities. A smoke-test script was also included to execute random actions in the environment and ensure that everything runs properly.

Overall, by the end of Phase 1, we completed all core logic, mechanics, and interactive visualization components of the game. The environment behaves correctly, the perfect-square manager and special abilities work as required, and the GUI provides a fully functional way to play and observe the system.